

From Six Months to Three Weeks

How a Developer and AI Built the Windows 98 Portfolio OS

The short version

The Windows 98 Portfolio OS is a browser-based operating-system simulation: a full boot sequence, a virtual C: drive, two dozen working applications, in-browser DOS games, a command-line shell, and a real crash-and-recovery model. Built alone, a project of this scope and polish would realistically take a single developer six to seven months. Built by one developer working with an AI pair, it came together in three to four weeks. This document explains how: the engineering patterns, the writing discipline, the real obstacles, and why the speed only appears in skilled hands.

The engineering patterns we used

The speed did not come from cutting corners. It came from a small set of disciplined patterns that kept the system testable and easy to extend:

- **Functional core, imperative shell.** Pure logic modules (filesystem, commands, recovery, audio, network) return plain data; React performs the side effects. The logic is testable without a browser.
- **One reducer state machine.** A single OS-state reducer owns every phase and window transition, so the whole system is explicit and inspectable.
- **Type-first contracts.** A shared types file defines the app IDs, window payloads, filesystem nodes, and the OS facade. Literal-union types and a tagged (discriminated) union for the Start-menu model let independent apps talk to the shell without inventing private protocols.
- **Effect descriptors.** The terminal returns effect objects (open an app, replace the filesystem, crash) instead of touching React directly: pure logic, real side effects.
- **Data-driven registries.** Adding an app, a game, or a media file is mostly a data edit, not a rewrite.

The writing style and conventions

Consistency was treated as a feature, because consistent code is faster to extend safely:

- New code reads like the code around it: matching naming, comment density, and idiom, including the codebase's own bilingual inline notes.
- Every change was verified before moving on. The type-checker, linter, and test suite were kept green at each step, ending at twenty-two passing tests.
- Honesty over hand-waving. When something could not be verified, it was reported as unverified rather than claimed as done.
- Small, reviewable commits with short, clear messages.

The hardships we faced

This was not a smooth ride. The speed came from working through genuine obstacles quickly, not from avoiding them:

- **The locked games.** Embedding DOOM and Wolfenstein from a public archive hit a cross-origin overlay that cannot be dismissed from an embedding page. The fix was an architecture pivot: self-host the games on js-dos, a DOSBox emulator compiled to WebAssembly.
- **The broken bundle.** The self-hosted game bundles failed twice. The cause was subtle: the Windows built-in ZIP tool writes path separators as backslashes, but the bundle format requires forward slashes. They had to be repackaged with a different tool.
- **The cache that lied.** Even after the fix, the browser kept serving the broken copy, because the emulator caches bundles by URL. The cure was URL versioning to force a fresh fetch.
- **The silent game.** DOOM ran but produced no sound; the trimmed bundle was missing the configuration file that selects a sound device. Restoring it brought the audio back.
- **The keyboard that drove two things at once.** Arrow keys controlled both the game and the desktop icons until a focus-gated guard gave the active game exclusive keyboard control.
- **The test environment that could not be trusted.** The headless preview throttles the emulator so heavily that its audio under-runs to silence, a false negative. Rather than trust it, the audio path was locked down with a unit test and confirmed by the developer in a real browser.

Each of these is the kind of multi-hour rabbit hole that, faced solo and unfamiliar, easily becomes a multi-day one.

Why AI boosts productivity, in the right hands

AI did not replace the developer. It amplified one, and the division of labor is the entire point:

- **The AI compressed the build loop.** It already knew the ZIP specification, browser autoplay rules, the emulator internals, DOSBox sound configuration, and how to hand-write WAV and BMP encoders, and it wrote and re-wrote code tirelessly.
- **The developer compressed the decision loop.** Choosing to abandon the iframe approach for self-hosting, designing the environment-variable privacy model for personal media, recognizing that the throttled preview was lying, and verifying the real result.

The phrase "in the right hands" is not flattery; it is the mechanism. Several times the AI's first answer was plausible but wrong: the stale cached bundle, the falsely silent audio. A non-developer would have shipped those. The developer caught them, redirected, and the AI executed the correction in minutes. Skill sets the direction and the bar; AI collapses the distance to it.

The timeline

Estimated solo effort, building and debugging all of this alone: six to seven months.

Actual effort with AI and one developer: three to four weeks.

Roughly a six-to-eight-times compression, not because the work got smaller, but because the loop between an idea and verified, working code got dramatically shorter.

The 4,000-line refactor

The clearest single proof point: a roughly four-thousand-line TypeScript refactor completed with zero regressions, with the help of a developer steering it. A monolithic stylesheet was split into per-component sheets and the OS shell was reorganized, while a type-checked build, a clean linter, and a full passing test suite confirmed that nothing broke. Refactors at that scale are exactly where solo developers lose days to subtle, silent breakage. Here it was done methodically and verified continuously.

The takeaway

AI is a force multiplier, and the multiplier scales with the operator. Put it in the hands of a developer who can architect, judge, and verify, and a half-year build becomes a three-week sprint, without lowering the bar on correctness. That is the story of this project.